

Claude Code 实战课程（中文翻译）

课程说明： 本文档由自动化脚本抓取生成

原课程链接： <https://anthropic.skilljar.com/claude-code-in-action/303233>

官方网站： <https://cholf5.com/claude-code-in-action/index.html>

课程目录

1. [引言](#)
2. [什么是编码助手？](#)
3. [Claude Code 实战](#)
4. [Claude Code 安装与配置](#)
5. [项目准备](#)
6. [添加上下文](#)
7. [进行修改](#)
8. [课程满意度调查](#)
9. [控制上下文](#)
10. [自定义命令](#)

- 11. [Claude Code 的 MCP 服务器](#)
 - 12. [GitHub 集成](#)
 - 13. [认识 Hooks](#)
 - 14. [定义 Hooks](#)
 - 15. [实现一个 Hook](#)
 - 16. [Hooks 常见坑点](#)
 - 17. [实用的 Hooks](#)
 - 18. [另一个实用 Hook](#)
 - 19. [Claude Code SDK](#)
 - 20. [Claude Code 测验](#)
 - 21. [总结与下一步](#)
-

引言

本节为课程引言视频。

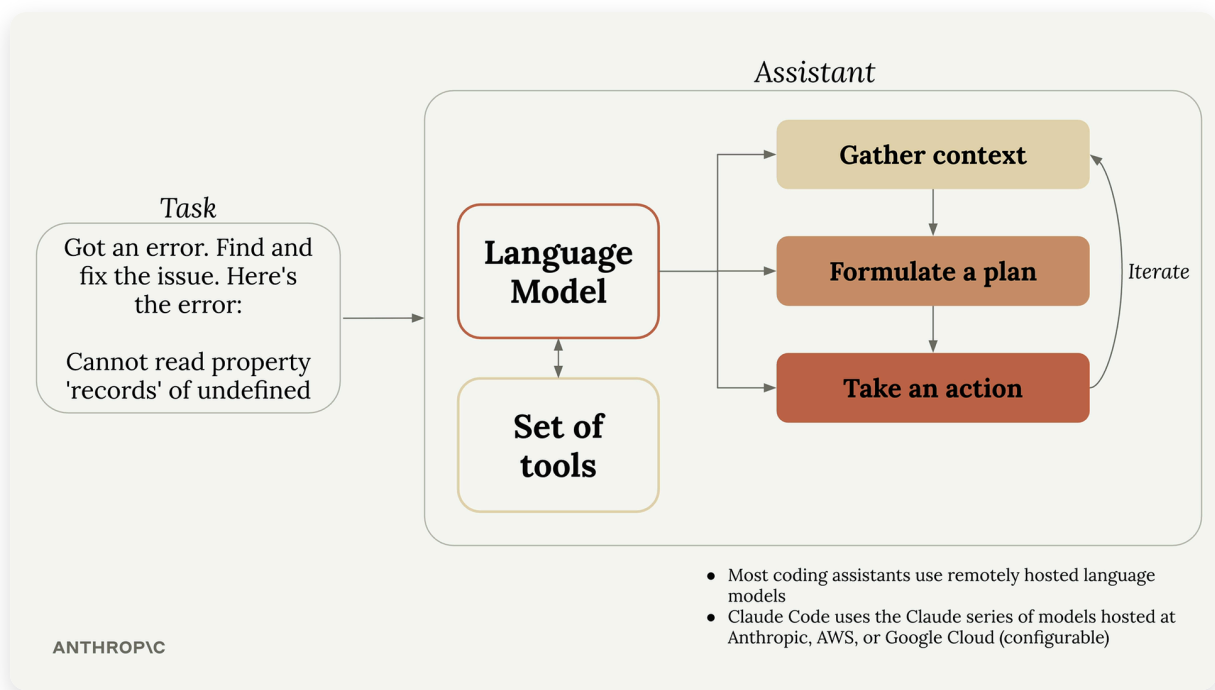
当前页面不包含文字讲义，请在原课程页面观看视频内容。

什么是编码助手？

编码助手不仅仅是写代码的工具——它是一个使用语言模型来处理复杂编程任务的系统。了解它们在幕后如何运作，能帮助你理解什么才是真正强大的编码伙伴。

编码助手如何工作

当你给编码助手一个任务（例如根据报错修复 Bug），它会按类似人类开发者的方式来推进：



- **收集上下文**：理解错误指向什么、哪些文件受影响、哪些文件相关
- **制定计划**：决定如何解决问题，例如修改代码并运行测试验证
- **采取行动**：真正去修改文件、运行命令并完成修复

关键洞见在于：第一步和最后一步都需要与外部世界交互——读文件、查文档、运行命令、编辑代码等。

工具使用的挑战

语言模型本身只能处理文本、输出文本，无法真正读取文件或运行命令。如果你直接让一个独立语言模型去读文件，它会告诉你自己没有这个能力。

那编码助手如何解决？它们使用一种巧妙的系统，叫作“工具使用”。

工具使用如何运作

当你向编码助手发送请求时，它会自动在消息中加上一些指令，教模型如何请求动作。比如它可能加上：“如果你想读文件，请回复 'ReadFile: 文件名'”。

完整流程如下：

- 你提问：“main.go 文件里写了什么代码？”
- 编码助手为你的请求添加工具指令
- 语言模型回应：“ReadFile: main.go”
- 编码助手读取真实文件内容并回传给模型
- 语言模型基于文件内容给出最终答案

这套机制让语言模型“看起来”能够读文件、写代码、运行命令——实际上它只是生成了格式化的文本响应。

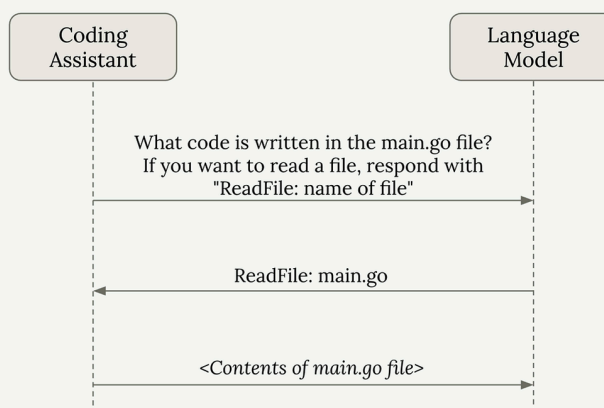
为什么 Claude 的工具使用很关键

不是所有语言模型都擅长使用工具。Claude 系列模型（Opus、Sonnet、Haiku）在理解工具、调用工具方面 尤其强。

Tool Use

- Models are given plain text directions on how to respond a certain way to use a 'tool'
- When the model responds with a request to use a tool, the coding assistant does whatever the tool is supposed to do (read a file, write a file, make a request, etc)
- **The Claude series of models (Opus, Sonnet, Haiku) are particularly strong at understanding what tools do and using them to complete tasks**

ANTHROPIC



强工具使用带来的好处包括：

强工具使用的收益

- **更难的任务也能完成**：Claude 能组合多种工具，甚至使用从未见过的新工具
- **平台可扩展**：你可以轻松为 Claude Code 增加新工具，Claude 会自适应你的流程
- **更好的安全性**：无需索引代码库即可导航，避免将整个代码库发送到外部服务器

要点回顾

理解编码助手的关键在于：

- 编码助手通过语言模型完成任务
- 语言模型需要工具才能处理真实世界的编程问题
- 不同模型的工具使用能力差异很大
- Claude 的工具使用能力提升了安全性、可定制性与长期可用性

正是这种工具使用能力，将一个只会生成文本的模型，转变成能读文件、理解代码库并实际修改项目的强大 编码助手。

Claude Code 实战

Claude Code 内置了一整套开发工具，涵盖读取文件、编写代码、运行命令、管理目录等常见任务。真正让 Claude Code 强大的是它能智能地组合这些工具，处理复杂的多步骤问题。

本节主要通过视频演示这些能力的实际使用方式。

Claude Code 安装与配置

准备开始在本机安装 Claude Code!

完整的安装说明请参考: <https://code.claude.com/docs/en/quickstart>

简要步骤如下:

- 安装 Claude Code

macOS (Homebrew) : `brew install --cask claude-code` macOS / Linux / WSL : `curl -fsSL https://claude.ai/install.sh | bash`

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && i
```

- 安装完成后, 在终端运行 `claude`。首次运行会提示你进行认证。

如果你使用 AWS Bedrock 或 Google Cloud Vertex, 还需要额外配置:

- AWS Bedrock 说明: <https://code.claude.com/docs/en/amazon-bedrock>
- Google Cloud Vertex 说明: <https://code.claude.com/docs/en/google-vertex-ai>

项目准备

有一个可以操作的项目, 会让你在 Claude Code 中练习时更有意思。

我准备了一个小项目供你探索, 它就是前面视频里演示的 UI 生成应用。注意: 你不一定需要运行这个项目, 如果你愿意, 也可以用自己的代码库跟随课程。

准备步骤

该项目需要一些基础设置：

- 确保本地安装了 Node.js。安装说明：<https://nodejs.org/en/download>
- 下载本节附带的 uigen.zip 并解压
- 在项目目录运行 npm run setup，安装依赖并初始化本地 SQLite 数据库
- **可选：**该项目使用 Anthropic API 调用 Claude 生成 UI 组件。如果你想完整体验应用，需要提供 API Key（不提供也能生成静态假代码）。

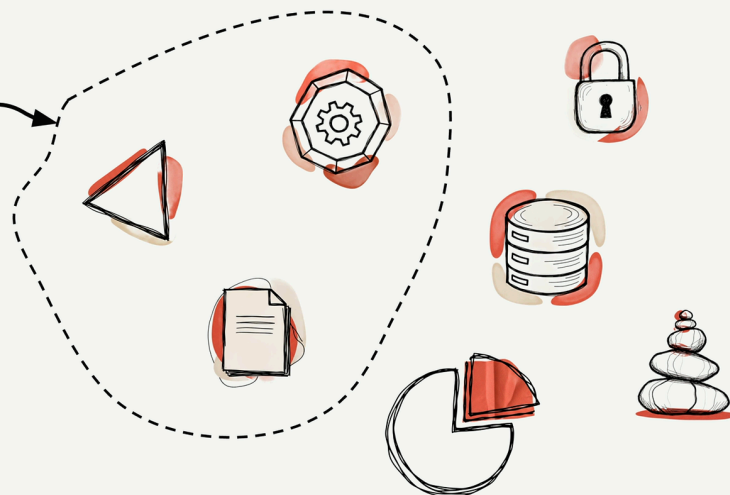
在
<https://console.anthropic.com/>
获取 API Key

将 API Key 写入 .env 文件 - 运行 npm run dev 启动项目

添加上下文

在用 Claude 处理编程项目时，上下文管理非常关键。你的项目可能有几十甚至上百个文件，但 Claude 只需要与任务相关的部分。过多无关上下文反而会降低 Claude 的表现，因此学会引导它定位关键文件与文档非常重要。

What Claude
needs to know to
accomplish your task



ANTHROPIC

/init 命令

当你在新项目里第一次启动 Claude 时，运行 /init 命令。它会分析整个代码库并理解：

- 项目目标与架构
- 关键命令与核心文件
- 代码风格与模式

The /init command

Scans your codebase

Creates a summary

Writes the summary to the
CLAUDE.md file

This file is included in every
request

```
> /init is analyzing your codebase...

• I'll analyze your codebase to understand its structure
  and create a comprehensive CLAUDE.md file. Let me
  start by exploring the repository.

* Jiving... (6s · 68 tokens · esc to interrupt)

> █
? for shortcuts
```

分析完成后，Claude 会生成一份摘要并写入 CLAUDE.md。当 Claude 询问是否允许写入时，你可以按 Enter 逐次确认，或按 Shift+Tab 让 Claude 在本次会话中自由写文件。

CLAUDE.md 文件

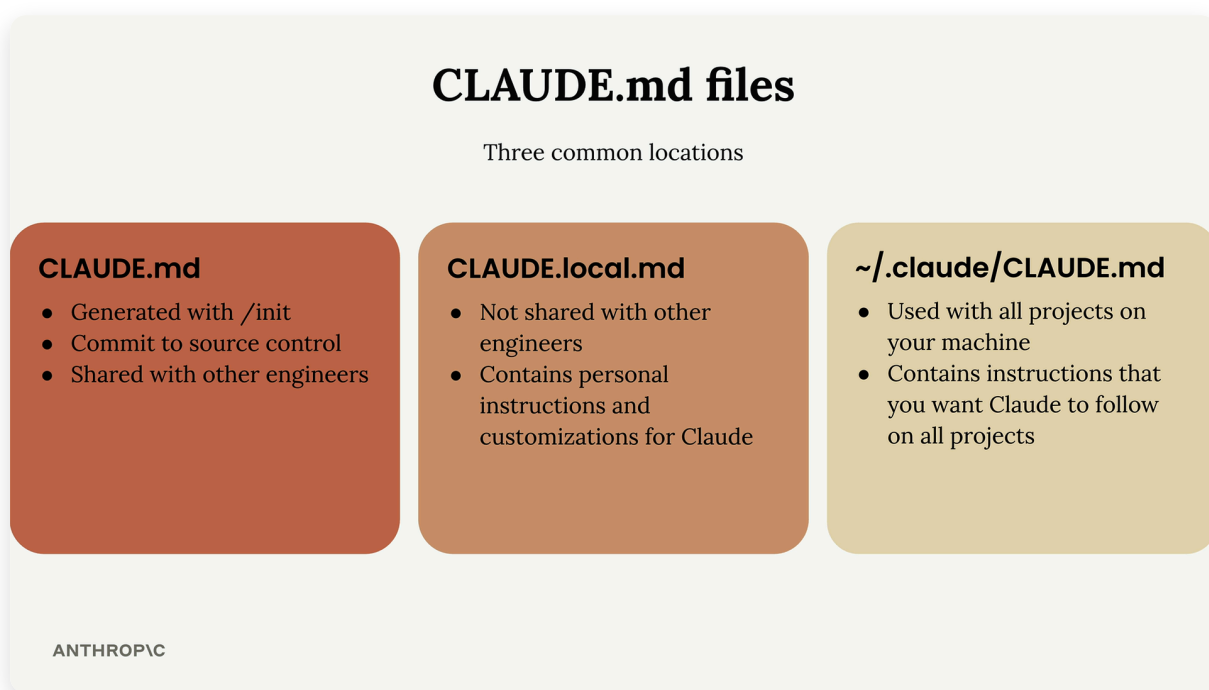
CLAUDE.md 有两个主要作用：

- 引导 Claude 理解你的代码库：重要命令、架构、代码风格
- 允许你给 Claude 添加特定或自定义指令

该文件会自动包含在每一次请求中，相当于项目级的持久系统提示词。

CLAUDE.md 的位置

Claude 识别以下三处常见位置的 CLAUDE.md：



- **CLAUDE.md**：由 `/init` 生成，提交到仓库，与团队共享
- **CLAUDE.local.md**：个人专用，不与团队共享
- **~/.claude/CLAUDE.md**：全局文件，适用于本机所有项目

添加自定义指令

你可以在 CLAUDE.md 中添加指令来调整 Claude 的行为。

使用 # 命令进入“记忆模式”，例如：

```
# Use comments sparingly. Only comment complex code.
```

Claude 会自动将这条指令合并进 CLAUDE.md。

使用 @ 提及文件

当你希望 Claude 查看某个文件时，可以用 @ 加上路径。这样会自动把该文件内容加入请求。

例如：

```
How does the auth system work? @auth
```

Claude 会列出相关文件供你选择，然后把选中的文件加入对话。

在 CLAUDE.md 中引用文件

你也可以在 CLAUDE.md 里用 @ 直接引用文件。

比如数据库 schema 文件：

```
The database schema is defined in the @prisma/schema.prisma file. Referenc
```

这样一来，每次请求都会自动包含该文件内容，Claude 就不需要反复搜索与读取。

进行修改

在开发环境中使用 Claude 时，你经常需要对现有项目进行修改。本节介绍了一些实用技巧：包括用截图进行精准沟通，以及利用 Claude 的高级推理能力提升修改质量。

用截图进行精准沟通

截图是与 Claude 沟通 UI 修改需求最有效的方式之一。想修改界面某个细节时，截图能让 Claude 准确理解你指的具体区域。

将截图粘贴到 Claude 中，请使用 Ctrl+V（macOS 不是 Cmd+V）。粘贴后，你就可以让 Claude 针对这部分进行修改。

规划模式（Planning Mode）

当任务比较复杂、需要在代码库中大量探索时，可以启用规划模式。它会让 Claude 先深入浏览项目，再提出实施方案。

按 Shift + Tab 两次即可启用（若已开启自动接受编辑，按一次即可）。在该模式中，Claude 会：

- 阅读更多项目文件
- 给出详细的实施计划
- 明确说明将要做的具体操作
- 在执行前等待你的批准

这样你可以先审阅计划，如果遗漏重要点或方向不对，可以及时引导。

思考模式 (Thinking Modes)

Claude 提供多个“思考”模式，让它在复杂问题上投入更多推理资源：

- Think：基础推理
- Think more：扩展推理
- Think a lot：深入推理
- Think longer：更长时间推理
- Ultrathink：最高强度推理

每个模式都会分配更多 token，用于更深层的分析与推理。

何时用规划模式 vs 思考模式

这两者针对不同类型的复杂度：

规划模式适合：

- 需要广泛理解代码库的任务
- 多步骤实施
- 涉及多个文件或组件的改动

思考模式适合：

- 复杂逻辑问题
- 疑难 Bug 排查
- 算法或推理挑战

如果一个任务既需要“广度”又需要“深度”，可以同时启用两种模式。但需要注意，它们都会消耗更多 token，因此会带来成本。

课程满意度调查

此页面为在线满意度调查，需要在原课程平台中填写。

静态版本无法展示交互问卷，请点击原文链接完成调查。

控制上下文

处理复杂任务时，你经常需要引导对话保持聚焦。下面这些技巧可以帮助你控制对话流，避免 Claude 走偏。

用 Esc 中断 Claude

当 Claude 开始偏离方向或一次性处理过多任务时，你可以按 Esc 中断它的响应，随后重新明确目标。

例如你让 Claude 为多个函数写测试，它可能开始规划整套测试体系。此时按 Esc，中断后让它先写一个函数的测试。

Esc + 记忆的组合

Esc 的一个强大用途是修复重复性错误：

- 按 Esc 停止当前回复
- 用 # 添加一条记忆（正确的做法）
- 继续对话，让 Claude 按新记忆执行

这样可以避免 Claude 在未来对话中重复同样的错误。

回退对话

长对话容易积累大量无关上下文。例如排错过程可能对下一任务无用。此时可以按 Esc 两次“回退对话”：

- 保留有价值的上下文（例如对代码库的理解）
- 删除无用或干扰性的对话内容
- 让 Claude 专注于当前任务

上下文管理命令

Claude 提供了几个专门管理上下文的命令：

/compact

/compact 会总结整个对话并保留关键要点。适用于：

- Claude 已学习到项目的重要信息
- 你要继续相关任务但希望对话更短
- 对话变长但仍有价值信息需要保留

/clear

/clear 会清空对话上下文，适用于：

- 切换到完全不相关的新任务
- 旧上下文可能干扰新任务
- 需要彻底重来

何时使用这些技巧

这些控制技巧特别适用于：

- 长对话导致上下文杂乱

- 任务切换时担心上下文干扰
- Claude 重复犯错
- 复杂项目需要保持聚焦

灵活使用 Esc、中断回退、/compact 与 /clear，可以让 Claude 在开发流程中保持高效与专注。这些不是小技巧，而是高质量 AI 开发会话的基础能力。

自定义命令

Claude Code 内置了一批以斜杠开头的命令，你也可以创建自己的命令，把常见流程自动化。

创建自定义命令

在项目中准备以下目录结构：

- 找到项目中的 .claude 目录
- 在其中创建 commands 目录
- 创建一个以命令名命名的 Markdown 文件（如 audit.md）

文件名就是命令名，因此 audit.md 会生成 /audit 命令。

示例：审计依赖的命令

一个实用的命令是检查依赖安全问题：

- 运行 npm audit 找出漏洞
- 运行 npm audit fix 自动修复
- 运行测试验证修复不破坏功能

创建命令文件后，需要重启 Claude Code 才能识别新命令。

带参数的命令

自定义命令可以使用 `$ARGUMENTS` 占位符接收参数，从而更灵活。

例如 `write_tests.md`：

```
Write comprehensive tests for: $ARGUMENTS
```

```
Testing conventions:
```

- * Use Vitests with React Testing Library
- * Place test files in a `__tests__` directory in the same folder as the source
- * Name test files as `[filename].test.ts(x)`
- * Use `@/` prefix for imports

```
Coverage:
```

- * Test happy paths
- * Test edge cases
- * Test error states

调用方式：

`/write_tests the use-auth.ts file in the hooks directory`

参数可以是任意文字说明，不一定是文件路径。

关键收益

- **自动化**：把重复流程变成一个命令
- **一致性**：确保每次执行遵循相同步骤
- **上下文**：为 Claude 提供固定的项目约定
- **灵活性**：通过参数适配不同场景

自定义命令非常适合项目内的固定流程，例如测试、部署、代码生成等。

Claude Code 的 MCP 服务器

你可以通过 MCP（Model Context Protocol）服务器扩展 Claude Code 的能力。MCP 服务器可以在本地或远程运行，为 Claude 提供原本没有的新工具与新能力。

最常用的 MCP 服务器之一是 Playwright，它能让 Claude 控制浏览器，为 Web 开发流程带来巨大提升。

安装 Playwright MCP 服务器

在终端运行以下命令（不要在 Claude Code 里运行）：

```
claude mcp add playwright npx @playwright/mcp@latest
```

该命令会：

- 把 MCP 服务器命名为 playwright
- 指定本地启动服务器的命令

权限管理

默认情况下 Claude 每次使用 MCP 工具都会请求权限。如果你不想频繁确认，可以在设置中预先允许：

```
{
  "permissions": {
    "allow": ["mcp__playwright"],
    "deny": []
  }
}
```

注意 `mcp__playwright` 中有双下划线。这样 Claude 便可直接使用 Playwright 工具。

实战示例：提升组件生成质量

Playwright 可以让 Claude 自动化以下流程：

- 打开浏览器并进入你的应用
- 生成测试组件
- 分析视觉样式与代码质量
- 更新生成提示词
- 再次测试新提示词

例如：

“访问 localhost:3000，生成一个基础组件，检查样式，然后更新 @src/lib/prompts/generation.tsx 里的提示词，让后续组件更好。”

Claude 会用浏览器工具观察真实视觉结果，再改写提示词，让生成的设计更有创意与差异化。

收益与效果

实践中，这种流程能显著提升生成质量，例如：

- 从“紫蓝渐变 + 标准 Tailwind 结构”升级为更丰富的配色
- 暖色夕阳渐变（橙 → 粉 → 紫）
- 海洋深度主题（青绿 → 翡翠 → 青蓝）
- 非对称布局与重叠元素
- 更具创造性的留白与结构

核心优势是：Claude 能看到真实视觉输出，而不是只盯着代码。

探索更多 MCP 服务器

Playwright 只是其中一个例子。MCP 生态还包括：

- 数据库交互
- API 测试与监控
- 文件系统操作
- 云服务集成
- 开发工具自动化

选择符合你需求的 MCP 服务器，可以让 Claude 从“代码助手”升级为“全流程开发伙伴”。

GitHub 集成

Claude Code 提供官方 GitHub 集成，让 Claude 在 GitHub Actions 中运行。主要有两个流程：在 Issue/PR 中 @Claude，以及自动 PR Review。

安装与配置

在 Claude 中运行 `/install-github-app`，它会引导你完成：

- 安装 Claude Code GitHub App
- 添加 API Key
- 自动生成包含 workflow 文件的 PR

合并该 PR 后，`.github/workflows` 中会出现两个 Actions。

默认的 GitHub Actions

Mention Action

在 Issue 或 PR 中使用 @claude, Claude 将：

- 分析任务并给出计划
- 以完整权限执行任务
- 在 Issue/PR 中回复结果

Pull Request Action

每次创建 PR 时，Claude 会自动：

- 审查改动
- 分析影响范围
- 发布详细评审报告

自定义工作流

合并初始 PR 后，你可以按项目需要调整工作流。

添加项目准备步骤

```
- name: Project Setup
  run: |
    npm run setup
    npm run dev:daemon
```

添加自定义指令

```
custom_instructions: |
  The project is already set up with all dependencies installed.
  The server is already running at localhost:3000. Logs from it
  are being written to logs.txt. If needed, you can query the
  db with the 'sqlite3' cli. If needed, use the mcp__playwright
  set of tools to launch a browser and interact with the app.
```

MCP 服务器配置

```
mcp_config: |
  {
    "mcpServers": {
      "playwright": {
        "command": "npx",
        "args": [
          "@playwright/mcp@latest",
          "--allowed-origins",
          "localhost:3000;cdn.tailwindcss.com;esm.sh"
        ]
      }
    }
  }
```

工具权限

在 GitHub Actions 中必须明确列出允许的工具（尤其是 MCP 工具）：

```
allowed_tools: "Bash(npm:*),Bash(sqlite3:*),mcp__playwright__browser_snaps
```

不同于本地环境，Actions 中没有快捷许可，必须逐项列出。

最佳实践

- 从默认工作流开始，逐步定制
- 用自定义指令补充项目上下文
- 使用 MCP 时务必写清工具权限
- 先用简单任务验证工作流，再升级复杂任务

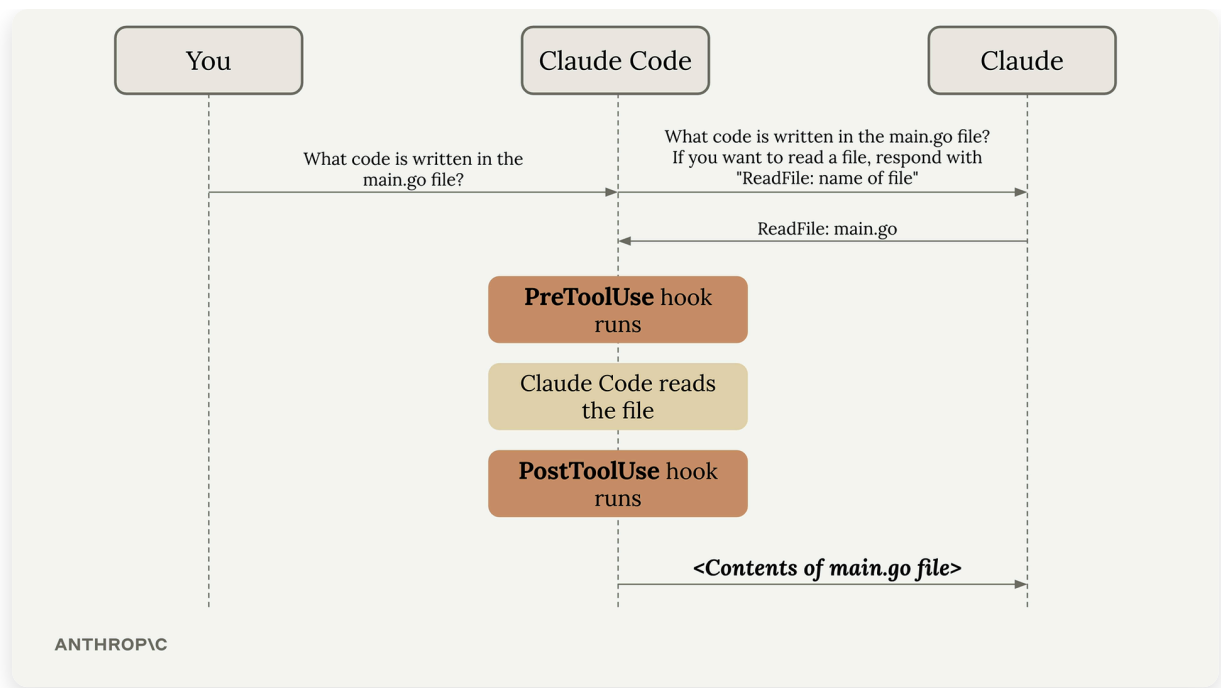
GitHub 集成让 Claude 从“开发助手”升级为团队中的自动化成员，可直接在 GitHub 流程里完成任务与评审。

认识 Hooks

Hooks 允许你在 Claude 使用工具前后运行自定义命令。它非常适合做自动化，比如在编辑后自动格式化代码、运行测试，或阻止访问特定文件。

Hooks 如何工作

在常规流程中，Claude 接收你的问题，决定使用工具，然后 Claude Code 执行工具并把结果返回给模型。Hooks 会插入到这个流程中，让你在工具执行前或后运行自己的逻辑。



Hooks 分为两类：

- **PreToolUse**：在工具执行前触发
- **PostToolUse**：在工具执行后触发

Hooks 配置位置

Hooks 写在 Claude 的设置文件中，可放在：

- **全局**：~/.claude/settings.json（影响所有项目）
- **项目级**：.claude/settings.json（团队共享）
- **项目级（不提交）**：.claude/settings.local.json（个人配置）

也可以在 Claude Code 内使用 `/hooks` 命令进行设置。

Hook Definitions

Defined in...

Global	~/.claude/settings.json
Project	.claude/settings.json
Project (not committed)	.claude/settings.local.json

Write **by hand** or by using the **/hooks** command

ANTHROPIC

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/read_hook.ts"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/edit_hook.ts"
          }
        ]
      }
    ]
  }
}
```

配置结构大致如下：

Hooks that run **before** a tool is called

Hooks that **after** before a tool is called

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/read_hook.ts"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/edit_hook.ts"
          }
        ]
      }
    ]
  }
}
```

ANTHROPIC

PreToolUse 示例

```
"PreToolUse": [  
  {  
    "matcher": "Read",  
    "hooks": [  
      {  
        "type": "command",  
        "command": "node /home/hooks/read_hook.ts"  
      }  
    ]  
  }  
]
```

该配置会在执行 Read 工具前运行指定命令，你可以：

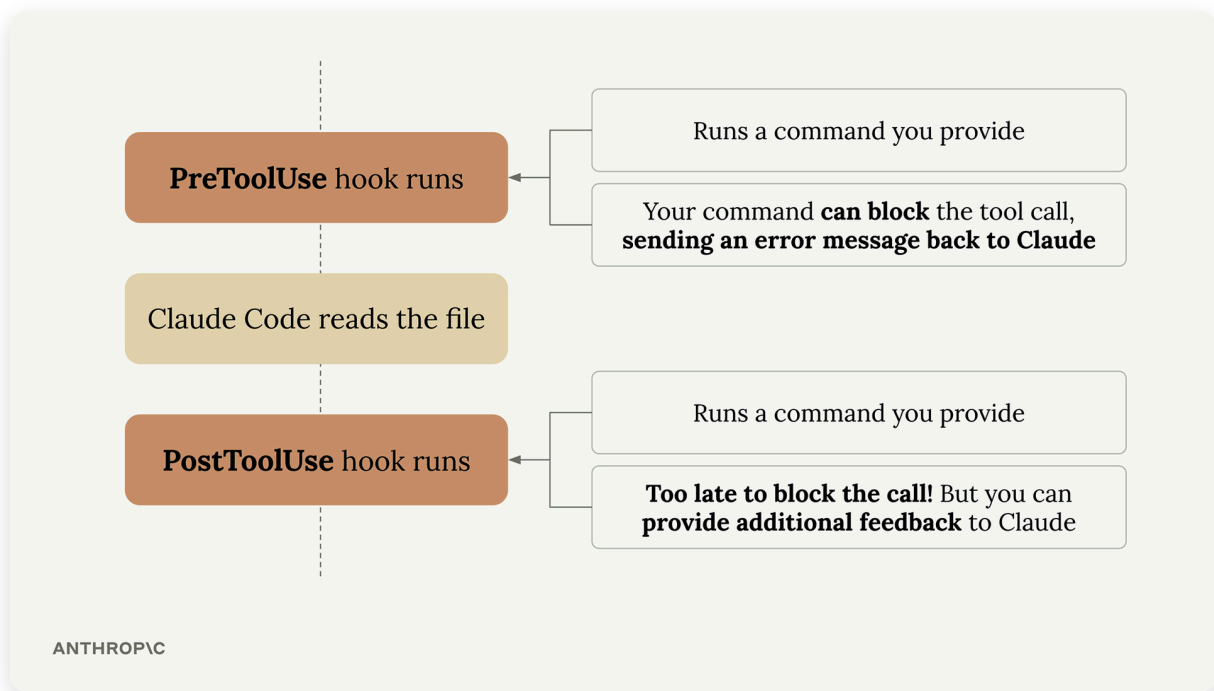
- 允许工具正常执行
- 阻止操作，并向 Claude 返回错误信息

PostToolUse 示例

```
"PostToolUse": [  
  {  
    "matcher": "Write|Edit|MultiEdit",  
    "hooks": [  
      {  
        "type": "command",  
        "command": "node /home/hooks/edit_hook.ts"  
      }  
    ]  
  }  
]
```

PostToolUse 无法阻止工具执行，但可以：

- 在编辑后自动运行格式化或测试
- 把额外反馈返回给 Claude



常见应用场景

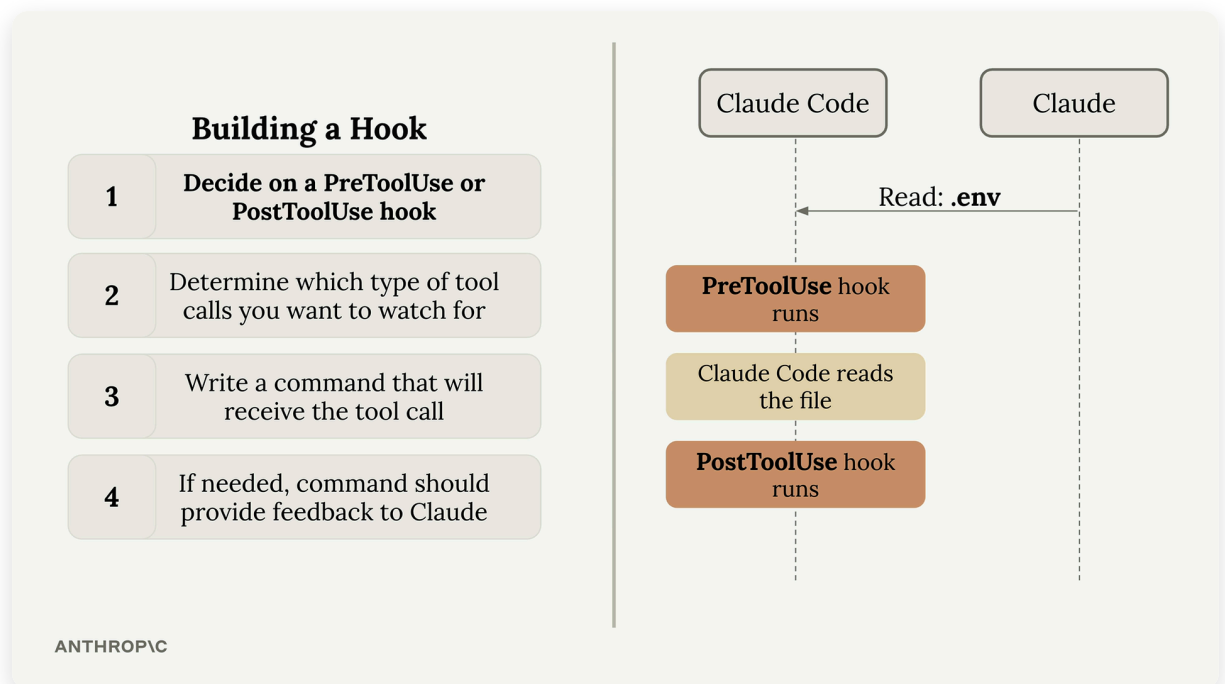
- **代码格式化**：编辑后自动格式化
- **自动测试**：文件变更后运行测试
- **访问控制**：阻止读写敏感文件
- **代码质量**：跑 linter/类型检查并反馈
- **日志记录**：追踪 Claude 访问的文件
- **规则校验**：强制命名或编码规范

Hooks 能把你的工具和流程整合进 Claude Code。PreToolUse 给你控制权，PostToolUse 让你增强 Claude 的结果。

定义 Hooks

Hooks 让你在工具调用前后拦截并控制 Claude 的行为，从而对开发环境拥有更细粒度的掌控。

构建一个 Hook 的步骤



- **选择 PreToolUse 或 PostToolUse：**前者可阻止工具执行，后者只能在执行后处理
- **确定要监控的工具类型：**明确哪些工具触发 Hook
- **编写接收工具调用的命令：**通过标准输入获取 JSON 数据
- **必要时向 Claude 反馈：**用退出码控制允许/阻止

可用工具

Building a Hook

1

Decide on a PreToolUse or PostToolUse hook

2

Determine which type of tool calls you want to watch for

3

Write a command that will receive the tool call

4

If needed, command should provide feedback to Claude

ANTHROPIC

Tool Names

Tool Name	Purpose
Read	Read a file
Edit, MultiEdit	Edit an existing file
Write	Create a file and write to it
Bash	Execute a command
Glob	Find files/folders based upon a pattern
Grep	Search for content
Task	Create a sub-agent to complete a particular task
WebFetch, WebSearch	Search or fetch a particular page

可用工具会随着 MCP 服务器变化，因此可直接让 Claude 列出当前工具列表以确认。

工具调用的数据结构

Building a Hook

1

Decide on a PreToolUse or PostToolUse hook

2

Determine which type of tool calls you want to watch for

3

Write a command that will receive the tool call

4

If needed, command should provide feedback to Claude

ANTHROPIC

Tool Call Data

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

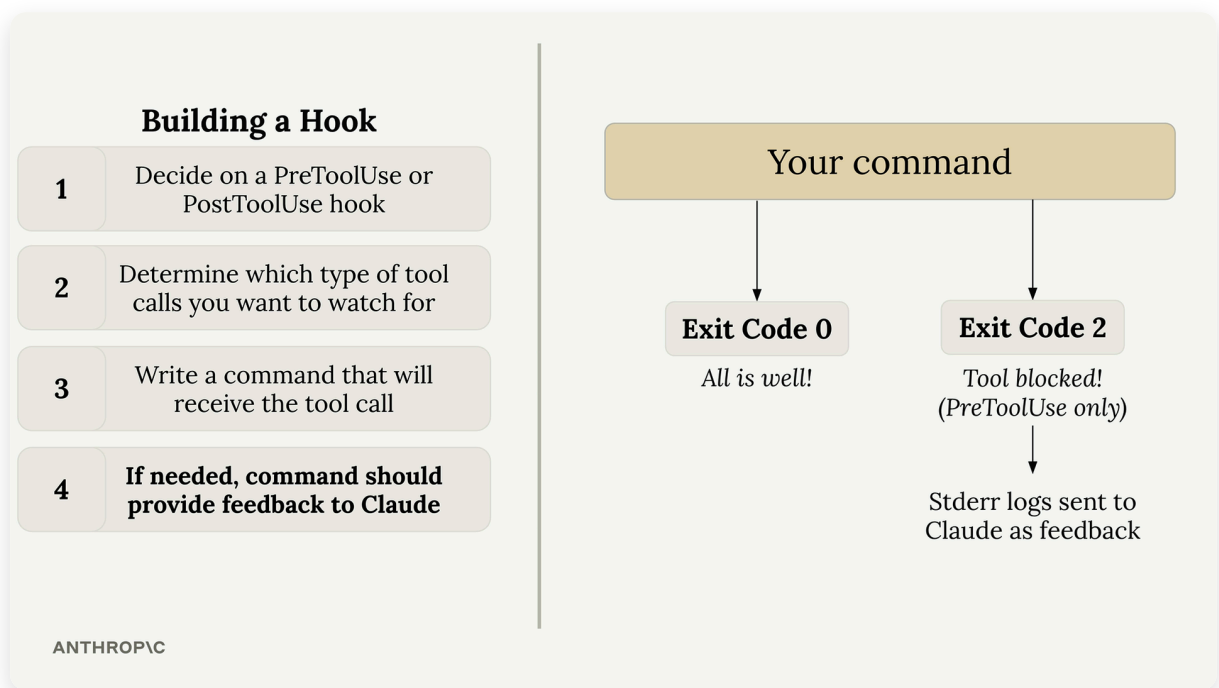
Standard In

Your command

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

Hook 命令读取该 JSON，并决定是否允许当前工具调用。

退出码与控制逻辑



- **退出码 0**：允许工具正常执行
- **退出码 2**：阻止工具执行（仅 PreToolUse 有效）

如果你在 PreToolUse 中返回 2，写到标准错误的内容会被 Claude 当作反馈信息。

常见示例

最常见的用途是阻止 Claude 读取敏感文件，例如 .env。因为 Read 和 Grep 都可能访问文件内容，你需要同时监控这两个工具，并检测是否指向敏感路径。

这样既能保护文件系统，又能清晰告诉 Claude 为什么被拦截。

实现一个 Hook

我们来构建一个 Hook，防止 Claude 读取敏感文件（例如 .env）。这是一个保护环境变量的典型场景。

配置 Hook

在 .claude/settings.local.json 中添加 PreToolUse Hook，用于在工具执行前拦截。

配置的关键要素包括：

- **matcher**：匹配触发的工具
- **command**：运行的脚本

示例：

```
"matcher": "Read|Grep"
```

管道符 | 表示“或”，因此 Read 或 Grep 都会触发。

```
"command": "node ./hooks/read_hook.js"
```

理解工具调用数据

Hook 通过标准输入接收 JSON，其中包含：

- 会话 ID 与 transcript 路径
- Hook 事件名 (PreToolUse)
- 工具名 (Read、Grep 等)
- 工具输入 (文件路径等)

你的脚本读取 JSON 后，决定允许或阻止。

实现 Hook 脚本

核心逻辑如下：

```
async function main() {
  const chunks = [];
  for await (const chunk of process.stdin) {
    chunks.push(chunk);
  }

  const toolArgs = JSON.parse(Buffer.concat(chunks).toString());

  // Extract the file path Claude is trying to read
  const readPath =
    toolArgs.tool_input?.file_path || toolArgs.tool_input?.path || "";

  // Check if Claude is trying to read the .env file
  if (readPath.includes('.env')) {
    console.error("You cannot read the .env file");
    process.exit(2);
  }
}
```

当路径包含 `.env` 时，脚本写错误并以退出码 2 终止，Claude 会理解这是 Hook 的阻止。

测试 Hook

保存配置与脚本后，重启 Claude Code，再尝试让 Claude 读取 `.env`。

Hook 会拦截并返回错误信息，Claude 会解释操作被 Hook 阻止。同理，如果 Claude 用 Grep 搜索 `.env`，也会被阻止。

收益

- **主动防护**：在敏感数据被读取前阻止
- **透明可解释**：Claude 会收到清晰的阻止原因
- **灵活匹配**：可覆盖多个工具与路径
- **可扩展**：适用于任意敏感文件/目录

你可以在此基础上扩展更多规则，实现更精细的访问控制。

Hooks 常见坑点

你可能注意到，在运行 `npm run dev` 后，`.claude` 目录里会出现两个 `settings.json` 文件。下面解释原因。

Claude Code 文档对 Hook 安全有一些推荐：

Security Best Practices

Here are some key practices for writing more secure hooks:

1. **Validate and sanitize inputs** - Never trust input data blindly
2. **Always quote shell variables** - Use `"$VAR"` not `$VAR`
3. **Block path traversal** - Check for `..` in file paths
4. **Use absolute paths** - Specify full paths for scripts
5. **Skip sensitive files** - Avoid `.env` , `.git/` , keys, etc.

其中一条是：脚本尽量使用绝对路径，而不是相对路径。这样可以降低 路径拦截 或 二进制植入 的风险。

但绝对路径也带来共享困难，因为你机器上的路径与其他人可能完全不同。

为了解决这个问题，项目提供了 `settings.example.json`。其中脚本路径使用 `$PWD` 占位符。运行 `npm run setup` 时，会执行 `scripts` 目录中的 `init-claude.js`：

- 将 `$PWD` 替换为本机项目的绝对路径
- 复制 `settings.example.json`
- 重命名为 `settings.local.json`

这样既能共享配置，又能满足使用绝对路径的安全建议。

实用的 Hooks

Claude Code 的 Hooks 能弥补 AI 协作中的常见问题，尤其在大型项目里效果明显。它们会在 Claude 修改代码时 自动执行，提供即时反馈并阻止常见错误。

TypeScript 类型检查 Hook

Claude 修改函数签名时，经常忘记更新所有调用点。例如为 `schema.ts` 中函数添加 `verbose` 参数后，`main.ts` 的调用还保持旧签名，导致类型错误。

解决办法是使用 `PostToolUse` Hook，在每次编辑后运行 TypeScript 编译器：

- 运行 `tsc --noEmit` 做类型检查
- 收集错误
- 将错误反馈给 Claude
- 提示 Claude 修复相关文件

对于其他强类型语言，也可以使用类似的类型检查流程；弱类型语言则可改用自动化测试。

防止重复查询的 Hook

在有大量数据库查询的项目中，Claude 有时会重复造轮子。例如你要求它“增加一个超过三天未处理订单的 Slack 提醒”，它可能重新写查询而不是复用 `getPendingOrders()`。

该 Hook 的思路是加入“二次审查”流程：

- 当 Claude 修改 `./queries` 目录下的文件时触发
- 程序化启动另一个 Claude Code 实例
- 让第二个实例检查是否已有相似查询
- 若发现重复，反馈给原 Claude
- 提示删除重复代码并复用现有实现

实现注意点

TypeScript Hook 较轻量，速度快；查询重复 Hook 更消耗资源，因为它会启动额外 Claude 实例。

- **收益：**减少重复代码，提升一致性
- **成本：**每次修改都要额外调用，耗时并消耗 API

- **建议：**只监控关键目录，避免过度开销

这些 Hook 使用 Claude 的 TypeScript SDK，通过编程方式让一个 Claude 去审查另一个 Claude 的输出。

可扩展思路

- 用编译器或 linter 输出做即时反馈
- 用独立 AI 实例做自动代码审查
- 重点监控高价值目录
- 权衡自动化收益与性能成本

关键是找出你流程中的痛点，并用 Hook 自动解决。

另一个实用 Hook

除 PreToolUse 与 PostToolUse 外，还有更多 Hook 类型：

- Notification：Claude 请求工具权限或 60 秒空闲时触发
- Stop：Claude 回复结束时触发
- SubagentStop：子代理任务结束时触发
- PreCompact：compact 操作前触发
- UserPromptSubmit：用户提交提示词时触发
- SessionStart：会话开始或恢复时触发
- SessionEnd：会话结束时触发

令人困惑的地方在于：

- 不同 Hook 的标准输入结构完全不同
- PreToolUse/PostToolUse 的输入还会随工具类型变化

例如，下面是一个 PostToolUse（监听 TodoWrite）的输入：

```
{
  "session_id": "9ecf22fa-edf8-4332-ae85-b6d5456eda64",
  "transcript_path": "<path_to_transcript>",
  "hook_event_name": "PostToolUse",
  "tool_name": "TodoWrite",
  "tool_input": {
    "todos": [{ "content": "write a readme", "status": "pending", "priority": 1 }],
    "oldTodos": [],
    "newTodos": [{ "content": "write a readme", "status": "pending", "priority": 1 }],
    "oldTodos": []
  }
}
```

而 Stop Hook 的输入是：

```
{
  "session_id": "af9f50b6-f042-4773-b3e2-c3a4814765ce",
  "transcript_path": "<path_to_transcript>",
  "hook_event_name": "Stop",
  "stop_hook_active": false
}
```

可以看到，不同 Hook 的输入差异非常大，这使得编写 Hook 变得困难——你不一定知道该解析哪些字段。

建议做一个辅助 Hook 来记录输入：

```
"PostToolUse": [ // Or "PreToolUse" or "Stop", etc
  {
    "matcher": "*",
    "hooks": [
      {
        "type": "command",
        "command": "jq . > post-log.json"
      }
    ]
  },
]
```

该命令会把 Hook 输入写入 post-log.json，方便你观察真实结构，从而更容易编写稳定的 Hook。

Claude Code SDK

Claude Code SDK 让你可以在应用或脚本中以编程方式调用 Claude Code。它提供 TypeScript、Python 以及 CLI 方式，功能与终端中的 Claude Code 一致。

Claude Code SDK

- Available for Typescript, Python, and via the CLI
- Runs Claude Code programatically
- Same Claude Code!
- Inherits all the settings of an instance of Claude Code launched in the same directory
- Has read-only permissions by default
- Useful as part of a larger pipeline or tool

ANTHROPIC

CLI

```
claude -p "Look for duplicate queries"
```

Typescript

```
import { query, SDKMessage } from "@anthropic-ai/claude-code";  
  
const prompt = "Look for duplicate queries";  
  
for await (const message of query({prompt})) {  
  console.log(message);  
}
```

Python

```
import anyio  
from claude_code_sdk import query  
  
async def main():  
    prompt = "Look for duplicate queries"  
    async for message in query(prompt=prompt):  
        print(message)  
  
anyio.run(main)
```

SDK 运行的就是你熟悉的 Claude Code，同样具备完整工具集，适用于自动化与系统集成。

关键特性

- 支持编程方式调用 Claude Code
- 功能与终端版本一致
- 继承同目录下 Claude Code 的设置
- 默认只读权限
- 适合嵌入更大的自动化流程

基础用法

以下是一个 TypeScript 示例，用于查找重复查询：

```
import { query } from "@anthropic-ai/claude-code";

const prompt = "Look for duplicate queries in the ./src/queries dir";

for await (const message of query({
  prompt,
})) {
  console.log(JSON.stringify(message, null, 2));
}
```

运行后你会看到 Claude Code 与模型之间的完整消息流，最终消息即 Claude 的完整响应。

权限与工具

SDK 默认是只读模式，只能读取与检索文件，无法写入或编辑。如果需要写权限，可以在调用时传入 `allowedTools`：

```
for await (const message of query({
  prompt,
  options: {
    allowedTools: ["Edit"]
  }) {
  console.log(JSON.stringify(message, null, 2));
}
```

也可以在项目的 `.claude` 设置文件中进行全局授权。

实用场景

- 在 Git hooks 中自动评审改动

- 在构建脚本中分析和优化代码
- 辅助维护任务的工具命令
- 自动生成文档
- CI/CD 中的代码质量检查

SDK 让你把 AI 能力融入任意开发环节，是自动化与集成场景的强大基础设施。

Claude Code 测验

此页面为在线测验，需要在原课程平台中完成。

静态版本无法展示交互测验，请点击原文链接参与测试。

总结与下一步

本节为课程总结视频。

当前页面不包含文字讲义，请在原课程页面观看视频内容。
